

课程笔记

CS224N Lecture 5: Recurrent Neural Networks

整理：AI Course Notes Contributors；授课：Chris Manning

2024 年

课程版本：2024 年中文讲义整理版

笔记来源：AI Course Notes Contributors

本版处理：移除课件截图和对应图注，不包含 Stanford 原始课件、图片或视频。

授权：笔记依 CC BY-NC-SA 4.0 分享；完整许可与改动记录见下一页。

来源与许可

本中文笔记由 AI Course Notes Contributors 整理，课程版本为 2024；源稿：<https://github.com/hqhq1025/ai-course-notes/tree/717e2df65c3d2eee593c171156a30d4f8f6812b9/cs224n/lecture05>。

笔记内容依 CC BY-NC-SA 4.0 分享：<https://creativecommons.org/licenses/by-nc-sa/4.0/>。

本副本的改动：移除 Stanford 原始课件截图与依赖这些截图的图注，移除课程视频封面/链接，并清理 TeX 源稿中的行尾空白后重新编译为可独立阅读的 PDF；同时加入本来源与许可说明。完整许可文本随本文件夹提供。Stanford 课件、课件图片和视频不包含在该授权内；请从对应年份 Stanford 课程页访问原始课件。

目录

1 引言与深度学习基础回顾	3
1.1 从浅层到深层：深度学习的崛起	3
1.2 正则化：从经典到现代视角	3
1.2.1 经典正则化	3
1.2.2 现代视角：Double Descent	3
1.2.3 Dropout：现代神经网络的标配正则化	4
1.3 向量化：深度学习的性能基石	4
1.4 参数初始化	4
1.5 优化器：从 SGD 到 Adam	4
1.6 本章小结	5
2 语言模型	5
2.1 什么是语言模型	5
2.2 语言模型无处不在	5
2.3 N-gram 语言模型	6
2.3.1 基本思想	6
2.3.2 N-gram 模型的问题	6
2.3.3 用 N-gram 模型生成文本	6
2.4 固定窗口神经语言模型	6
2.5 本章小结	7
3 循环神经网络 (RNN)	7
3.1 RNN 的核心思想	7
3.2 RNN 的数学定义	7
3.3 训练 RNN 语言模型	8
3.3.1 损失函数	8
3.3.2 Teacher Forcing	8
3.3.3 实际训练中的分段处理	9
3.4 反向传播：BPTT	9
3.4.1 重复权重的梯度	9
3.4.2 截断反向传播	9
3.5 用 RNN 生成文本	9
3.6 RNN 的优势与缺陷	10
3.7 本章小结	10
4 RNN 的梯度问题	10
4.1 梯度消失与梯度爆炸	10
4.2 梯度消失的后果	11
4.3 梯度爆炸的后果与解决方案	11
4.4 梯度消失的解决思路	11
4.5 本章小结	12

5 RNN 的应用场景	12
5.1 序列标注	12
5.2 文本分类与情感分析	12
5.3 条件语言模型与编码器-解码器架构	12
5.4 本章小结	12
6 总结与延伸	13
6.1 全课知识图谱	13
6.2 关键 Takeaways	13
6.3 展望：从 RNN 到 Transformer	13
6.4 拓展阅读	14

1 引言与深度学习基础回顾

本节课是 Stanford CS224N 的第五讲，由 Chris Manning 教授主讲。课程内容分为三个主要部分：(1) 深度学习实用技巧的补充（正则化、参数初始化、优化器等）；(2) 语言模型（Language Models）的概念与构建方法；(3) 循环神经网络（Recurrent Neural Networks, RNN）的原理与应用。

1.1 从浅层到深层：深度学习的崛起

Manning 教授首先回顾了神经网络的发展历史。在 1980–1990 年代，虽然反向传播算法已经被 Hinton 等人推广，但当时几乎所有的神经网络都只有一个隐藏层。原因在于，人们长期无法让深层网络比浅层网络表现更好。

深度学习的历史转折

直到 2006 年前后，Yoshua Bengio 等人提出逐层贪心预训练（Greedy Layer-Wise Training）等方法，深度网络才开始逐步展现出优于浅层网络的能力。Manning 指出，推动这一转变的并非某个惊天发现，而是若干“小想法”和“小技巧”的积累——正则化方法、激活函数选择、参数初始化策略、优化器改进等。这些看似不起眼的改进，让一个停滞了 15 年的领域焕发新生。

1.2 正则化：从经典到现代视角

1.2.1 经典正则化

正则化（Regularization）是通过在损失函数中添加额外约束来防止过拟合的技术。最常见的形式是 L2 正则化：

$$J(\theta) = J_{\text{data}}(\theta) + \lambda \sum_i \theta_i^2$$

- $J_{\text{data}}(\theta)$: 数据拟合项（如交叉熵损失）
- $\lambda \sum_i \theta_i^2$: 正则化项，鼓励参数保持较小的值
- λ : 正则化强度超参数

1.2.2 现代视角：Double Descent

Manning 特别强调，现代大规模神经网络的行为与经典理论**截然不同**。在足够大的网络中，即使模型完全记住了训练数据（训练损失趋近于零），验证损失仍然会继续下降。

现代正则化的范式转变

现代大型神经网络可以将训练数据几乎完美拟合（训练损失接近零），这在经典统计学看来是灾难性的过拟合。但实际上，只要正则化做得好，模型仍然能很好地泛化到新数据。这一发现被称为“Double Descent”现象，彻底改变了我们对模型复杂度和泛化的理解。

1.2.3 Dropout: 现代神经网络的标配正则化

传统的 L2/L1 正则化对大型神经网络往往不够强。Manning 介绍了 **Dropout**——一种专为神经网络设计的正则化方法。

Dropout 的核心机制

1. **训练时**: 对每个训练样本, 随机生成一个由 0 和 1 组成的掩码 (mask), 通过 Hadamard 积将部分神经元的输出置零。每次使用不同的掩码。
2. **测试时**: 不进行 Dropout, 保留所有权重, 但对输出进行缩放以补偿训练时的丢弃。

Dropout 为什么有效? Manning 给出了三种直觉解释:

- **防止特征共适应** (Feature Co-adaptation): 模型不能过度依赖某几个特征的组合, 因为它们随时可能被删除
- **隐式模型集成**: 每次 Dropout 相当于训练一个不同的子网络, 最终效果类似于指数级数量的模型集成
- **介于 Naive Bayes 和 Logistic Regression 之间**: Naive Bayes 独立地评估每个特征, Logistic Regression 在所有特征的上下文中设置权重, Dropout 则在随机子集的上下文中学习, 提供了一种中间地带

1.3 向量化: 深度学习的性能基石

永远不要写 for 循环

深度学习的成功很大程度上依赖于向量化计算。在 Python 中, 使用 for 循环逐元素处理数据比向量/矩阵操作慢一到两个数量级。在 GPU 上, 这个差距更是达到两到三个数量级。如果你发现自己在写 for 循环处理数据 (而不是最表层的输入预处理), 几乎一定是做错了。

1.4 参数初始化

神经网络的参数初始化至关重要。如果将所有参数初始化为零或同一常数, 网络会陷入**对称性陷阱**——所有神经元学到的东西完全相同, 相当于只有一个神经元。

Xavier 初始化

Xavier/Glorot 初始化设置权重的方差为:

$$\text{Var}(W_i) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

其中 n_{in} 是前一层的大小, n_{out} 是后一层的大小。这确保信号在前向和反向传播过程中既不会消失也不会爆炸。Manning 指出, 后来 Layer Normalization 的引入在很大程度上消除了对精确初始化的需求。

1.5 优化器: 从 SGD 到 Adam

Manning 简要介绍了优化器的发展:

- **SGD** (随机梯度下降): 最基础的优化器, 理论上能解决几乎所有问题, 但对学习率和调度非常敏感
- **Adagrad**: 为每个参数维护历史梯度平方和, 自适应调整学习率。缺点是学习率会过早衰减
- **Adam**: 结合动量和自适应学习率, 是最常用的默认选择

如果只记住一个优化器

Adam 是最安全的默认选择。它对超参数不敏感, 几乎在所有场景下都能给出不错的结果。对于词向量等稀疏更新场景, 带有权重衰减的变体 (AdamW) 可能更合适。

1.6 本章小结

- 深度学习的成功建立在若干关键技巧的积累之上: Dropout 正则化、合理的参数初始化、自适应优化器
- 现代大型网络的过拟合行为与经典理论不同——Double Descent 现象表明, 足够大的模型即使完美拟合训练数据仍能良好泛化
- 向量化计算是深度学习效率的基石, 应避免在数据处理中使用 for 循环
- Adam 优化器是实践中最可靠的默认选择

2 语言模型

2.1 什么是语言模型

语言模型的定义

语言模型 (Language Model, LM) 是一个能够对文本序列中下一个词的出现概率进行预测的系统。形式化地, 给定前文 $x^{(1)}, x^{(2)}, \dots, x^{(t)}$, 语言模型输出下一个词的条件概率分布:

$$P(x^{(t+1)} \mid x^{(1)}, x^{(2)}, \dots, x^{(t)})$$

该概率分布满足 $\sum_{w \in V} P(w \mid x^{(1)}, \dots, x^{(t)}) = 1$, 其中 V 是词汇表。

等价地, 语言模型可以为任意一段文本 $x^{(1)}, \dots, x^{(T)}$ 赋予一个概率。通过链式法则:

$$P(x^{(1)}, \dots, x^{(T)}) = P(x^{(1)}) \cdot P(x^{(2)} \mid x^{(1)}) \cdot P(x^{(3)} \mid x^{(1)}, x^{(2)}) \cdots P(x^{(T)} \mid x^{(1)}, \dots, x^{(T-1)})$$

$$= \prod_{t=1}^T P(x^{(t)} \mid x^{(1)}, \dots, x^{(t-1)})$$

2.2 语言模型无处不在

Manning 强调, 语言模型并非 ChatGPT 时代的发明。它的历史可以追溯到 1950 年代 Claude Shannon 的信息论工作, 至少从 1980 年代起就已经是 NLP 的核心技术。

2.3 N-gram 语言模型

在神经网络语言模型出现之前（约 1975–2012 年），**N-gram 语言模型**是主流方法。

2.3.1 基本思想

N-gram 模型做了一个**马尔可夫假设** (Markov Assumption)：预测下一个词时，只使用前 $n - 1$ 个词作为上下文，丢弃更早的信息。

$$P(x^{(t+1)} \mid x^{(1)}, \dots, x^{(t)}) \approx P(x^{(t+1)} \mid x^{(t-n+2)}, \dots, x^{(t)})$$

概率估计通过计数得到：

$$P(w \mid \text{students opened their}) = \frac{\text{count}(\text{students opened their } w)}{\text{count}(\text{students opened their})}$$

2.3.2 N-gram 模型的问题

N-gram 模型的两大致命缺陷

1. **稀疏性问题** (Sparsity)：很多合理的 n-gram 组合在训练语料中从未出现过，导致概率估计为零。而概率为零是灾难性的——任何涉及该 n-gram 的计算都会变为零。解决方法包括加法平滑 (Laplace Smoothing) 和回退 (Backoff) 策略。
2. **存储问题** (Storage)：需要存储语料中所有观察到的 n-gram 的计数。增大 n 或增大语料规模都会使存储需求指数级增长。实践中 n 通常不超过 5。

2.3.3 用 N-gram 模型生成文本

尽管 N-gram 模型很“粗糙”，但它已经能生成看起来有点像样的文本。Manning 展示了用 tri-gram 模型在 200 万词语料上生成的文本：

Scale 的故事古已有之

Manning 有一个精彩的观察：今天人们说的“更多数据、更大模型就能解决一切”，与 2010 年代初人们对 N-gram 模型的态度如出一辙——当时人们也在说，只要有更大的语料和更高级的 N-gram，性能就会持续提升。但事实证明，**更好的模型架构**同样重要。

2.4 固定窗口神经语言模型

N-gram 模型的自然演进是使用神经网络来替代简单的计数。Yoshua Bengio 在 2003 年提出了**固定窗口神经语言模型** (Fixed-Window Neural Language Model)。

该模型的工作流程如下：

1. 将窗口内的每个词通过词嵌入矩阵 E 转换为向量 $e^{(i)}$
2. 将所有词嵌入拼接为一个长向量
3. 通过隐藏层： $h = f(W \cdot [e^{(1)}; e^{(2)}; \dots; e^{(n)}] + b_1)$
4. 通过输出层和 softmax： $\hat{y} = \text{softmax}(U \cdot h + b_2)$

固定窗口模型的优势与局限

优势：

- 无稀疏性问题——不需要存储所有 n-gram 计数
- 存储成本更低——只需存储网络参数
- 通过词嵌入实现了一定的泛化能力

局限：

- 仍然使用固定窗口，仍有马尔可夫假设的限制
- 不同位置的词使用权重矩阵 W 的不同部分处理，没有参数共享——同一个词出现在不同位置时，学到的知识无法迁移
- 窗口大小的选择是个难题——没有任何固定窗口是“足够大”的

2.5 本章小结

- 语言模型是预测下一个词的概率分布的系统，是 NLP 的核心技术
- N-gram 语言模型通过计数和马尔可夫假设工作，简单但受限于稀疏性和存储问题
- 固定窗口神经语言模型解决了稀疏性问题，但仍受限于固定上下文长度和缺乏参数共享
- 我们需要一种能处理任意长度输入且共享参数的新架构——这就引出了循环神经网络

3 循环神经网络 (RNN)

3.1 RNN 的核心思想

循环神经网络 (Recurrent Neural Network, RNN) 是一种专为处理序列数据设计的神经网络架构。其核心思想是：在每个时间步应用相同的权重矩阵，并维护一个隐藏状态 (hidden state) 来编码已处理过的所有历史信息。

3.2 RNN 的数学定义

在每个时间步 t ，RNN 执行以下计算：

$$h^{(t)} = \tanh(W_h \cdot h^{(t-1)} + W_e \cdot e^{(t)} + b)$$

- $h^{(t)} \in \mathbb{R}^{d_h}$ ：时间步 t 的隐藏状态
- $h^{(t-1)} \in \mathbb{R}^{d_h}$ ：前一时间步的隐藏状态
- $W_h \in \mathbb{R}^{d_h \times d_h}$ ：隐藏状态到隐藏状态的权重矩阵
- $e^{(t)} \in \mathbb{R}^{d_e}$ ：时间步 t 的词嵌入
- $W_e \in \mathbb{R}^{d_h \times d_e}$ ：输入到隐藏状态的权重矩阵

- $b \in \mathbb{R}^{d_h}$: 偏置项
- $\tanh$: 非线性激活函数 (RNN 中最常用的选择)

初始隐藏状态 $h^{(0)}$ 通常设为全零向量。

要将 RNN 用作语言模型，需要在每个时间步将隐藏状态映射到词汇表上的概率分布：

$$\hat{y}^{(t)} = \text{softmax}(U \cdot h^{(t)} + b_2)$$

RNN 的关键特性

1. **可处理任意长度的输入**：隐藏状态可以无限累积信息，不受固定窗口限制
2. **参数共享**： W_h 和 W_e 在每个时间步都相同——无论 “student” 出现在序列的什么位置，都会被同一套参数处理
3. **模型大小不随上下文增长**：无论输入多长，参数数量固定。只是计算量与序列长度成正比

为什么用 $\tanh$ 而不是 ReLU

传统 RNN 中通常使用 $\tanh$ 作为激活函数，而非在前馈网络中更常见的 ReLU。 $\tanh$ 的输出范围在 $[-1, 1]$ ，这在正负两侧是平衡的，有助于隐藏状态在多次递推后保持数值稳定。如果使用 ReLU（输出非负），隐藏状态可能会不断累加正值而爆炸。

3.3 训练 RNN 语言模型

3.3.1 损失函数

RNN 语言模型的训练目标是最小化每个时间步预测的交叉熵损失：

$$J^{(t)}(\theta) = -\log P(x^{(t+1)} \mid x^{(1)}, \dots, x^{(t)}) = -\log \hat{y}_{x^{(t+1)}}^{(t)}$$

总体损失是所有时间步损失的平均：

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$$

3.3.2 Teacher Forcing

训练时采用 **Teacher Forcing** 策略：在每个时间步，无论模型预测了什么词，都将**实际的下一个词**（来自训练语料）作为下一时间步的输入。

Teacher Forcing vs 自由生成

Teacher Forcing：训练时每步都“纠正”输入，使用真实的下一个词。优点是训练稳定高效；缺点是模型从未练习过从自己的错误中恢复。

自由生成 (Free Generation/Rollout)：只在推理时使用——将模型自己生成的词作为下一步的输入，持续生成直到产生终止符 $\langle /s \rangle$ 。这也是 ChatGPT 等系统生成回复的方式。

3.3.3 实际训练中的分段处理

理论上可以在整个语料上运行 RNN，但实际上这不可行——需要存储的中间状态太多。实际做法是将文本切分为固定长度的片段（如 100 个词），对每个片段独立计算损失和梯度。

分段引入了隐式马尔可夫假设

虽然 RNN 理论上能处理任意长度的上下文，但分段训练意味着模型实际上只能利用片段长度以内的上下文。Manning 指出，这在某种程度上又回到了马尔可夫假设——只是窗口比 N-gram 大得多（100 词 vs 3-5 词）。现代大语言模型通过使用更长的上下文窗口（数千个 token）来缓解这一限制。

3.4 反向传播：BPTT

3.4.1 重复权重的梯度

RNN 的一个关键特点是权重矩阵 W_h 在每个时间步都被使用。那么如何计算损失对 W_h 的梯度？

$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^t \frac{\partial J^{(i)}}{\partial W_h} \Big|_{(i)}$$

重复权重的梯度法则

对于重复使用的权重，其梯度等于该权重在每个位置出现时的梯度之和。数学上，这可以理解为：将 W_h 在每个时间步“虚拟地”视为不同的参数 $W_h^{(1)}, W_h^{(2)}, \dots$ ，分别计算梯度，然后求和。这遵循多变量链式法则中的分支求和规则。

3.4.2 截断反向传播

完整的反向传播通过时间（Backpropagation Through Time, BPTT）需要展开整个序列，计算代价很高。**截断 BPTT**（Truncated BPTT）是一种实用的近似：

- 前向传播仍然使用完整的上下文来更新隐藏状态
- 但反向传播只回溯固定的步数（如 20 步），然后截断
- 这大大减少了计算和存储开销，实践中通常足够好

3.5 用 RNN 生成文本

文本生成的流程：

1. 输入起始符号 $\langle s \rangle$ ，初始隐藏状态 $h^{(0)} = \mathbf{0}$
2. 通过 RNN 得到下一个词的概率分布
3. 从该分布中**采样**（sampling）一个词
4. 将采样得到的词作为下一时间步的输入

5. 重复步骤 2-4，直到采样到终止符 $\langle /s \rangle$

Manning 展示了几个有趣的生成示例：

字符级 RNN 与条件生成

RNN 不仅可以在词级别工作，还可以在**字符级别**逐字母生成文本。Manning 展示了一个有趣的应用：用颜色的 RGB 值初始化隐藏状态，然后让字符级 RNN 生成颜色名称（如 “Ghostly Pink”、“Power Gray” 等），效果出奇地好。这是**条件语言模型**（Conditional Language Model）的一个简单例子——用额外信息（颜色值）来引导生成。

3.6 RNN 的优势与缺陷

RNN 相比前代模型的进步

1. 可以处理**任意长度**的输入序列
2. **参数共享**：同一个词无论出现在什么位置，都被同样的权重处理
3. 模型大小不随上下文长度增长
4. 理论上能利用**任意远**的历史信息

RNN 的两大实际问题

1. **顺序计算，无法并行**：RNN 必须按时间步依次计算隐藏状态（本质上是一个 for 循环），无法利用 GPU 的并行计算能力。这使得 RNN 的训练速度远慢于后来的 Transformer。
2. **长距离依赖难以学习**：虽然理论上隐藏状态包含了所有历史信息，但实际上，随着序列变长，早期的信息会在隐藏状态中逐渐“衰减”，最近的词总是支配隐藏状态。这就是所谓的**梯度消失问题**，我们将在下一节详细讨论。

3.7 本章小结

- RNN 的核心思想是在每个时间步应用相同的权重，并通过隐藏状态编码历史信息
- 训练使用 Teacher Forcing 策略，损失函数为每步预测的交叉熵平均值
- 重复权重的梯度等于各时间步梯度之和；实践中常用截断 BPTT 加速
- RNN 能处理任意长度输入且参数共享，但受限于顺序计算和长距离依赖学习困难

4 RNN 的梯度问题

4.1 梯度消失与梯度爆炸

RNN 面临的核心训练难题是**梯度消失**（Vanishing Gradient）和**梯度爆炸**（Exploding Gradient）。考虑损失 $J^{(t)}$ 对早期隐藏状态 $h^{(1)}$ 的偏导数：

$$\frac{\partial J^{(t)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \cdots \times \frac{\partial h^{(t)}}{\partial h^{(t-1)}} \times \frac{\partial J^{(t)}}{\partial h^{(t)}}$$

每个 $\frac{\partial h^{(i)}}{\partial h^{(i-1)}}$ 涉及对权重矩阵 W_h 和激活函数导数的乘积。当这些矩阵的**谱范数**（最大奇异值）：

- 小于 1：连续相乘后梯度**指数级衰减** → **梯度消失**
- 大于 1：连续相乘后梯度**指数级增长** → **梯度爆炸**

4.2 梯度消失的后果

梯度消失意味着无法学习长距离依赖

当梯度消失时，来自远处时间步的梯度信号几乎为零。这意味着：

- 模型无法学到“远处的词对当前预测很重要”这样的模式
- 隐藏状态虽然理论上包含所有历史信息，但梯度信号无法告诉模型如何更好地利用早期信息
- 模型的有效上下文窗口远小于序列长度

4.3 梯度爆炸的后果与解决方案

梯度爆炸的解决方案相对简单——**梯度裁剪**（Gradient Clipping）：

$$\text{if } \|\nabla_{\theta} J\| > \tau : \quad \nabla_{\theta} J \leftarrow \frac{\tau}{\|\nabla_{\theta} J\|} \cdot \nabla_{\theta} J$$

- $\|\nabla_{\theta} J\|$ ：梯度的范数
- τ ：裁剪阈值（超参数）

梯度裁剪的直觉

梯度裁剪就像给 SGD 的更新步长设置了一个“速度限制”。它不改变梯度的方向（仍然朝着正确的方向走），只是在步子太大时把它缩小到安全范围内。这是一个简单但非常有效的技巧，在训练 RNN 时几乎是标配。

4.4 梯度消失的解决思路

梯度消失比梯度爆炸更难解决。Manning 提到了几个方向：

1. **LSTM**（Long Short-Term Memory）：通过门控机制（遗忘门、输入门、输出门）显式控制信息的保留和遗忘，是解决梯度消失最经典的方法
2. **GRU**（Gated Recurrent Unit）：LSTM 的简化版本，用更少的参数实现类似效果
3. **残差连接**（Residual Connections）：通过跳跃连接让梯度直接流过多层

这些内容将在后续课程中详细讲解。

4.5 本章小结

- 梯度消失和梯度爆炸源于反向传播中的矩阵连乘效应
- 梯度爆炸可以通过**梯度裁剪**有效解决
- 梯度消失更为棘手，导致 RNN 难以学习长距离依赖。主要解决方案包括 LSTM、GRU 等门控机制
- 这些问题最终推动了 Transformer 架构的诞生——通过自注意力机制直接建立任意位置之间的连接，彻底绕过了序列依赖问题

5 RNN 的应用场景

RNN 不仅可以用作语言模型，还广泛应用于各种序列处理任务。

5.1 序列标注

RNN 可以在每个时间步产生一个输出，用于**序列标注**（Sequence Tagging）任务：典型应用包括：

- **词性标注**（Part-of-Speech Tagging）：为每个词标注名词、动词、形容词等语法类别
- **命名实体识别**（Named Entity Recognition, NER）：识别人名、地名、组织名等

5.2 文本分类与情感分析

对于需要对整个序列产生单一判断的任务，可以使用 RNN 最后一个时间步的隐藏状态（或所有隐藏状态的某种聚合）作为序列的表示，再接分类层。

5.3 条件语言模型与编码器-解码器架构

通过用不同的信息初始化 RNN 的隐藏状态，可以构建**条件语言模型**：

从 RNN 到编码器-解码器

条件语言模型的思想自然延伸为**编码器-解码器**（Encoder-Decoder）架构：一个 RNN（编码器）读取输入序列并产生一个固定长度的隐藏表示，另一个 RNN（解码器）以该表示为条件生成输出序列。这一架构在机器翻译（Seq2Seq）中取得了巨大成功，将在后续课程中详细讨论。

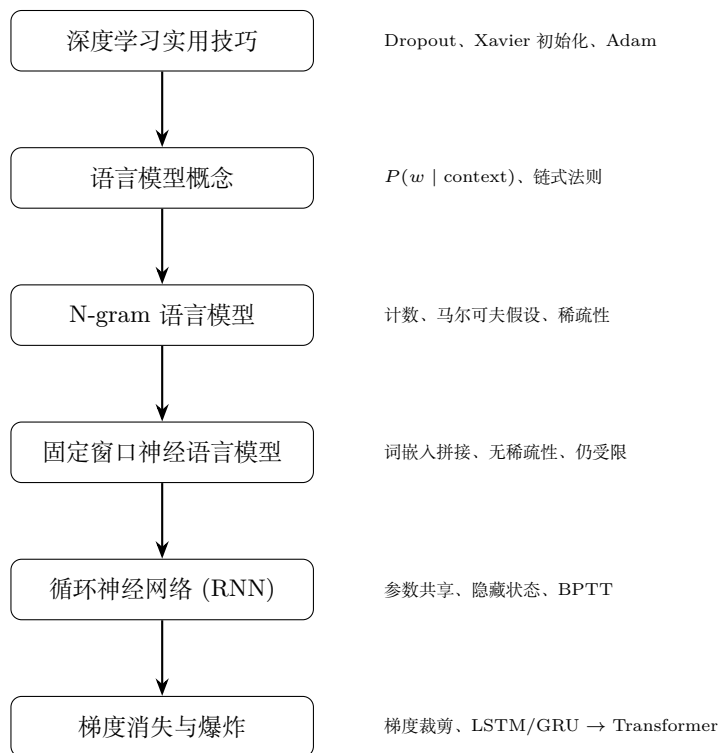
5.4 本章小结

- RNN 是一种通用的序列处理架构，适用于多种 NLP 任务
- 每步都有输出 → 序列标注（词性标注、命名实体识别）
- 只看最终输出 → 文本分类（情感分析）
- 用外部信息初始化 → 条件生成（语音识别、机器翻译）

6 总结与延伸

6.1 全课知识图谱

本课建立了从传统方法到神经网络方法的完整语言建模认知链：



6.2 关键 Takeaways

五条核心要点

1. **语言模型是 NLP 的基石**：从手机输入法到 ChatGPT，语言模型的核心任务始终是预测下一个词的概率分布
2. **从计数到学习**：N-gram 靠统计计数，神经语言模型靠参数学习——后者在泛化能力上有质的飞跃
3. **RNN 的核心创新是参数共享**：同一组权重在每个时间步重复应用，使模型能处理任意长度的输入
4. **梯度问题是序列模型的命门**：梯度消失限制了 RNN 学习长距离依赖的能力，这一问题推动了 LSTM、GRU 乃至 Transformer 的诞生
5. **实用技巧至关重要**：Dropout、合理初始化、Adam 优化器、梯度裁剪——这些“小技巧”的组合是深度学习成功的关键

6.3 展望：从 RNN 到 Transformer

Manning 在课程最后暗示了后续的发展方向：

- **LSTM 和 GRU**: 通过门控机制缓解梯度消失问题, 使 RNN 能更好地捕捉长距离依赖 (下一讲)
- **编码器-解码器架构**: 将 RNN 组合为翻译模型、摘要模型等序列到序列系统
- **注意力机制 (Attention)**: 让解码器在每步直接“看到”编码器的所有隐藏状态, 而非仅依赖最终状态
- **Transformer**: 完全抛弃循环结构, 用自注意力实现全并行计算, 彻底解决 RNN 的顺序计算瓶颈和长距离依赖问题

6.4 拓展阅读

- Bengio et al., “A Neural Probabilistic Language Model” (2003): <https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf> ——首个神经语言模型
- Mikolov et al., “Recurrent Neural Network based Language Model” (2010) ——RNN 语言模型的经典论文
- Hochreiter & Schmidhuber, “Long Short-Term Memory” (1997) ——LSTM 的原始论文
- Cho et al., “Learning Phrase Representations using RNN Encoder-Decoder” (2014) ——GRU 的提出
- Karpathy, “The Unreasonable Effectiveness of Recurrent Neural Networks” (2015): <http://karpathy.github.io/2015/05/21/rnn-effectiveness/> ——RNN 生成文本的有趣博客
- CS224N 课程主页: <https://web.stanford.edu/class/cs224n/>